

Avaliação 1 (Trabalho) de Ciência de Dados - NES

prof. Eduardo Adame

24 de abril de 2026

Objetivo da Avaliação

A AV1 consistirá em seminários individuais onde cada aluno apresentará um projeto prático integrando os conteúdos vistos ao longo do curso. O objetivo é demonstrar domínio das ferramentas de Ciência de Dados em um projeto coeso e funcional, com ênfase na **demonstração ao vivo** de uma aplicação rodando localmente.

Formato da Apresentação

- **Duração:** 10 minutos por apresentação
- **Data:** 08 de maio de 2026
- **Componentes obrigatórios:**
 - Slides com contextualização do projeto
 - Dashboard interativo rodando via **Docker** na máquina local
 - Demonstração ao vivo com processo gerador de dados
 - Código-fonte organizado em repositório Git

Cronograma

Data	Atividade
24/04/2026	Lançamento das instruções da AV1
01/05/2026	Prazo final para escolha do tópico
08/05/2026	Apresentações dos seminários

Atenção: A escolha do tópico deve ser feita na planilha até 01/05/2026. Atrasos resultarão em penalização na nota.

Arquitetura mínima obrigatória

Todo projeto deverá conter, obrigatoriamente:

- **docker-compose.yml** orquestrando todos os serviços necessários (banco de dados, aplicação, etc.)
- **Banco de dados persistente** (PostgreSQL, SQLite ou MySQL/MariaDB) com esquema definido
- **Processo gerador de dados** — um script Python que popula o banco com dados simulados ou reais
- **Dashboard interativo** que lê do banco e exibe visualizações (Streamlit, Dash, Gradio ou similar)
- **Repositório Git** com `README.md` descrevendo como rodar o projeto com `docker compose up`

O projeto deve rodar do zero com um único comando:

```
docker compose up
```

Tópicos Disponíveis

Cada aluno deverá escolher **um** dos seguintes tópicos. O tema define o **domínio dos dados** e a **pergunta analítica central** do projeto — a arquitetura técnica é a mesma para todos.

1. **Análise de desempenho escolar** Simule registros de alunos, turmas, notas e frequência ao longo de um semestre letivo. O dashboard deve permitir filtrar por turma, exibir distribuição de notas, evolução temporal da média e taxa de aprovação. Use o banco relacional com ao menos três tabelas relacionadas.
2. **Monitoramento de vendas de e-commerce** Simule transações de uma loja online com produtos, categorias, clientes e datas. O dashboard deve exibir faturamento por categoria, ticket médio, produtos mais vendidos e evolução de vendas por período. Inclua ao menos um gráfico temporal e uma tabela ranqueada.
3. **Análise de dados climáticos** Use dados históricos de temperatura, precipitação e umidade de estações meteorológicas (dados reais do INMET ou simulados). O dashboard deve mostrar séries temporais, médias por mês e comparativos entre estações ou cidades.
4. **Painel de saúde pública** Simule registros de atendimentos hospitalares com diagnósticos, faixa etária, bairro e data. O dashboard deve exibir incidência por região, evolução temporal de casos e distribuição por faixa etária. Use dados geoespaciais simples (bairros como categorias).
5. **Mercado financeiro simplificado** Use dados históricos de preços de ações (Yahoo Finance via `yfinance`) ou simule séries de preços com passeio aleatório. O dashboard deve exibir séries temporais, retornos diários, volatilidade e correlação entre ativos. Armazene os dados no banco e permita ao usuário selecionar o ativo e o período de análise.

6. **Análise de repositórios GitHub** Use a API pública do GitHub para coletar dados de repositórios de uma organização ou conjunto de usuários (estrelas, forks, linguagem, data de criação, commits recentes). O dashboard deve exibir distribuição de linguagens, atividade ao longo do tempo e um ranking de repositórios por engajamento. Armazene os dados coletados no banco.

Observação: Os tópicos serão atribuídos por ordem de escolha.

Critérios de Avaliação

A avaliação será composta por **6 critérios**, totalizando **10,0 pontos**, com possibilidade de **+0,5 pontos extras**:

1. Demonstração ao vivo (3,0 pontos)

Este é o critério de maior peso. O aluno deve mostrar o projeto funcionando na própria máquina no dia da apresentação, com o banco populado e o dashboard respondendo a interações em tempo real.

- **3,0 pontos:** Dashboard rodando via Docker, banco com dados, filtros funcionando
- **2,0 pontos:** Dashboard funcionando, mas sem Docker ou com problemas parciais
- **1,0 ponto:** Aplicação parcialmente funcional ou apenas demonstrada em vídeo
- **0,0 pontos:** Projeto não roda ou não é demonstrado

2. Qualidade do banco de dados (2,0 pontos)

- Esquema relacional com ao menos duas tabelas e chave estrangeira
- Uso correto de tipos de dados e constraints (NOT NULL, PRIMARY KEY)
- Consultas SQL não triviais alimentando o dashboard (JOINS, agregações, filtros)
- Conexão com Pandas ou Polars via SQLAlchemy

3. Processo gerador de dados (1,5 pontos)

- Script Python que popula o banco de forma reproduzível (semente fixa)
- Dados com distribuição realista para o domínio escolhido
- Código organizado, comentado e sem hardcoding de caminhos

4. Qualidade do dashboard (1,5 pontos)

- Ao menos três tipos de visualização distintos
- Filtros ou seletores interativos que alteram as visualizações
- Layout limpo, títulos informativos e eixos rotulados

5. Organização do repositório (1,0 ponto)

- `README.md` com instruções claras de execução
- `docker-compose.yml` funcional com serviços bem definidos
- Commits com mensagens descritivas; `.gitignore` adequado
- Estrutura de pastas lógica (`src/`, `data/`, `sql/`, etc.)

6. Entrega do tópico no prazo (1,0 ponto)

- **1,0 ponto:** Tópico escolhido até 01/05/2026
- **0,5 pontos:** Tópico escolhido entre 02/05 e 04/05/2026
- **0,0 pontos:** Tópico não escolhido ou escolhido após 04/05/2026

Bônus: +0,5 pontos extras se os slides forem produzidos em **LaTeX** / **Beamer** ou **Typst**

Entregáveis

No dia da apresentação, o aluno deverá:

1. Chegar com o **projeto rodando** (`docker compose up` já executado)
2. Submeter o **link do repositório Git** público (GitHub, GitLab ou similar)
3. O repositório deve conter:
 - `docker-compose.yml`
 - `README.md` com instruções de execução
 - Script gerador de dados
 - Código do dashboard
 - Slides em PDF

Não é necessário entregar relatório escrito — a demonstração ao vivo substitui essa etapa.

Dicas para uma Boa Apresentação

- Teste o `docker compose up` em um ambiente limpo (outra máquina ou após `docker compose down -v`) para garantir que funciona do zero
- Deixe o banco pré-populado antes da apresentação — não gere dados ao vivo para evitar imprevistos
- Prepare um roteiro de demonstração: quais filtros vai usar, quais gráficos vai destacar
- Streamlit e Dash são as opções mais simples para o dashboard; escolha o que você domina
- Para o banco, PostgreSQL via Docker é recomendado; SQLite com arquivo montado no container também funciona
- Cite as fontes dos dados reais se usar APIs ou datasets externos
- Conecte seu projeto ao conteúdo do curso: mencione Pandas, SQL, Parquet, EDA — mostre que você integrou o que foi aprendido